

NIA Website — MERN Stack

Folder Structure & Architecture Plan | Netherlands India Association

1. Project Context

Project	Netherlands India Association (NIA) Website
Phase	Starting with Home Page — designed to scale per client requirement
Frontend	React (Vite)
Backend	Node.js + Express
Database	MongoDB Atlas
Storage	AWS S3
Payments	Stripe
AI	Claude (Anthropic)
Deployment	Render

2. Folder Structure

```
NIA Website/
├── Docs/                                # existing – leave as-is
├── client/                              # React frontend (Vite)
│   ├── public/
│   │   └── favicon.ico
│   ├── src/
│   │   ├── assets/                    # images, fonts, icons
│   │   ├── components/
│   │   │   ├── layout/
│   │   │   │   ├── Navbar.jsx
│   │   │   │   ├── Footer.jsx
│   │   │   └── home/                 # Home-page components
│   │   │       ├── HeroBanner.jsx
│   │   │       ├── AboutSnippet.jsx
│   │   │       ├── UpcomingEvents.jsx
│   │   │       ├── MembershipCTA.jsx
│   │   │       └── NewsHighlights.jsx
│   └── pages/
```

```

■ ■ ■ ■ ■ HomePage.jsx      # composes home/ components
■ ■ ■ ■ ■ hooks/           # custom React hooks
■ ■ ■ ■ ■ services/
■ ■ ■ ■ ■ api.js           # axios wrappers
■ ■ ■ ■ ■ context/        # auth, theme providers
■ ■ ■ ■ ■ styles/
■ ■ ■ ■ ■ index.css
■ ■ ■ ■ ■ App.jsx          # router + layout wrapper
■ ■ ■ ■ ■ main.jsx         # Vite entry point
■ ■ ■ ■ ■ .env             # VITE_API_URL etc.
■ ■ ■ ■ ■ index.html
■ ■ ■ ■ ■ vite.config.js
■ ■ ■ ■ ■ package.json
■
■ ■ ■ ■ ■ server/         # Node / Express backend
■ ■ ■ ■ ■ src/
■ ■ ■ ■ ■ config/
■ ■ ■ ■ ■ db.js            # MongoDB Atlas connection
■ ■ ■ ■ ■ s3.js            # AWS S3 client
■ ■ ■ ■ ■ stripe.js        # Stripe client
■ ■ ■ ■ ■ models/         # Mongoose schemas
■ ■ ■ ■ ■ routes/
■ ■ ■ ■ ■ index.js         # mounts all routers
■ ■ ■ ■ ■ controllers/     # route handler logic
■ ■ ■ ■ ■ middleware/
■ ■ ■ ■ ■ auth.js          # JWT verification
■ ■ ■ ■ ■ errorHandler.js
■ ■ ■ ■ ■ services/       # claude.js, stripe.js, s3.js
■ ■ ■ ■ ■ utils/
■ ■ ■ ■ ■ .env             # MONGO_URI, AWS, STRIPE, ANTHROPIC keys
■ ■ ■ ■ ■ server.js        # Express entry point
■ ■ ■ ■ ■ package.json
■
■ ■ ■ ■ ■ .gitignore       # covers both client/ and server/ .env

```

3. Key Conventions

■ New Page Workflow	Add components/<section>/ folder → build components → compose in pages/<PageName>.jsx → add route in App.jsx
■ API Calls	All live in client/src/services/api.js using axios. Base URL sourced from VITE_API_URL environment variable.
■ No Shared Package	client/ and server/ are independent package.json workspaces — simpler for Render's separate web service + static site deployment model.

4. Implementation Steps

01	Scaffold Client	npm create vite@latest client -- --template react
02	Scaffold Server	npm init -y then install: express, mongoose, cors, dotenv, jsonwebtoken
03	Create Folder Skeleton	Create all directories above with empty index files as placeholders
04	Root .gitignore	Cover node_modules, .env, and dist/ for both client and server
05	Wire MongoDB	server/src/config/db.js — mongoose.connect() reading MONGO_URI from .env
06	Wire React Router	client/src/App.jsx — react-router-dom setup rendering HomePage
07	Build Home Components	HeroBanner, AboutSnippet, UpcomingEvents, MembershipCTA, NewsHighlights as static stubs

5. Verification Checks

Frontend	cd client && npm run dev	Vite dev server renders HomePage with all five sections visible
Backend	cd server && node server.js	Express starts and connects to MongoDB Atlas (or logs helpful error if MONGO_URI unset)
Isolation	— no cross-imports —	No cross-import between client/ and server/ directories

6. Running Locally

```
# Frontend
cd client && npm run dev      # → http://localhost:5173
# Backend (in a separate terminal)
cd server && npm run dev      # → http://localhost:5000
```



Before starting the server, fill in **server/.env** with your real credentials: **MONGO_URI**, **STRIPE_SECRET_KEY**, and **ANTHROPIC_API_KEY**.

7. Adding a New Page

Adding a new page later follows one simple repeatable pattern:

Step	Action	Detail
Step 1	Create folder	components/<section>/
Step 2	Build components	Write individual .jsx component files inside the new folder
Step 3	Compose page	Import and assemble components in pages/<PageName>.jsx
Step 4	Register route	Add a <Route> entry for the new page in App.jsx